

3 O Modelo de Dados Relacional e as Restrições de um Banco de Dados Relacional

Esta unidade inicia com uma abordagem sobre os bancos de dados relacionais. O modelo relacional foi introduzido por Ted Codd, da IBM Research, em 1970, em um artigo clássico (Codd, 1970) que imediatamente atraiu a atenção em virtude de sua simplicidade e base matemática. O modelo usa o conceito de uma *relação matemática*, algo como uma tabela de valores como seu bloco de construção básica e tem sua base teórica na teoria dos conjuntos e na lógica de predicados de primeira ordem. Neste item discutiremos as características básicas do modelo e suas restrições.

As primeiras implementações comerciais do modelo relacional tornaram-se disponíveis no início da década de 80, como o SGBD Oracle e o sistema SQL/DS do sistema operacional MVS, da IBM. Desde essa época, o modelo tem sido implementado em um grande número de sistemas comerciais. Os SGBDs relacionais (SGBDRs) mais conhecidos atualmente são o DB2 e o *Informix Dynamic Server* (da IBM), o *Oracle* e o *Rdh* (da *Oracle*), e o *SQL Server* e o *Access* (da *Microsoft*).

Em razão da importância do modelo relacional, dedicaremos toda as unidades restantes desta apostila a esse modelo e às linguagens associadas a ele. A unidade 4 cobre as operações de álgebra relacional e introduz a notação de cálculo relacional para dois tipos de cálculo o de tupla e o de domínio. Na unidade 5 descrevemos a linguagem de consulta SQL, que é o padrão para os SGBDs relacionais comerciais.

Os modelos de dados que precederam o modelo relacional compreendem os modelos hierárquico e de rede. Foram propostos na década de 60 e implementados em antigos SGBDs durante as décadas de 70 e 80.

Nesta unidade, concentraremos-nos na descrição dos princípios básicos do modelo relacional de dados. Começaremos definindo os conceitos de modelagem e a notação do modelo relacional, no item 3.1. O item 3.2 é dedicado à discussão de restrições relacionais, que são, agora, consideradas uma importante parte do modelo relacional, sendo automaticamente garantidas pela maioria dos SGBDs relacionais. O item 3.3 define as operações de atualização do modelo relacional e discute como as violações de restrições de integridade são tratadas.

3.1 Conceitos do Modelo Relacional

O modelo relacional representa o banco de dados como uma coleção de *relações*. Informalmente, cada relação se parece com uma tabela de valores ou, em alguma extensão, com um arquivo de registros 'plano'. Por exemplo, o banco de dados dos arquivos que foram mostrados no exemplo do item 1.2 da apostila, é similar à representação do modelo relacional. Entretanto, há importantes diferenças entre relações e arquivos, conforme veremos em breve.

Quando uma relação é pensada como urna tabela de valores, cada linha na tabela representa uma coleção de valores de dados relacionados. No modelo relacional, cada linha na tabela representa um fato que corresponde a uma entidade ou relacionamento do mundo real. O nome da tabela e os nomes das colunas são usados para ajudar na interpretação do significado dos valores em cada linha. Por exemplo, a primeira tabela do

exemplo apresentado no item 1.2 da apostila é chamada ALUNO, pois cada linha representa os fatos sobre uma entidade aluno em particular. Os nomes das colunas Nome, NumeroAluno, Classe e Curso_Hab, especificam como interpretar os valores de dados em cada linha, com base na coluna em que cada valor está. Todos os valores em uma coluna são do mesmo tipo de dado.

Na terminologia do modelo relacional formal, uma linha é chamada *tupla*, um cabeçalho de coluna é conhecido como *atributo*, e a tabela é chamada *relação*. O tipo de dado que descreve os tipos de valores que podem aparecer em cada coluna é representado pelo *domínio* de valores possíveis. Definimos, agora, esses termos: *domínio*, *tupla*, *atributo* e *relação*, mais precisamente.

3.1.1 Domínios, Atributos, Tuplas e Relações

Um **domínio** D é um conjunto de valores atômicos. Por **atômico** entendemos que cada valor no domínio é indivisível no que diz respeito ao modelo relacional. Um método comum para a especificação de um domínio é definir um tipo de dado do qual os valores de dados que formam o domínio sejam retirados. Também é útil especificar um nome para esse domínio, de modo a ajudar na interpretação de seus valores. Seguem alguns exemplos de domínios:

- Numeros_fone_EUA: o conjunto de números de telefone válido nos Estados Unidos, com dez dígitos.
- Numeros_fone_local: o conjunto de números de telefone de sete dígitos, válido para um código de área em particular nos Estados Unidos.
- Numeros_seguro_social: o conjunto de nove dígitos válidos do número do seguro social.
- Nomes: o conjunto de cadeias de caracteres que representa os nomes de pessoas.
- Medias_pontos_graduacao: possíveis valores de médias computadas de pontos para a graduação; cada um deve ser um número real (ponto-flutuante) entre zero e quatro.
- Idades_empregado: possíveis idades dos empregados de uma empresa; cada um deve ter um valor entre 15 e 80 anos de idade.
- Nomes_departamento_academico: o conjunto dos nomes dos departamentos acadêmicos em uma universidade, como ciência da computação, economia e física.
- Codigos_departamento_academico: o conjunto de códigos dos departamentos acadêmicos, como CC, ECON e FIS.

Os precedentes são chamados definições *lógicas* de domínios. Um **tipo de dado** ou **formato** também é especificado para cada domínio. Por exemplo, o tipo de dado para o domínio Numeros_fone_EUA pode ser declarado como uma cadeia de caracteres no formato $(ddd)ddd-dddd$, em que cada d é um dígito numérico (decimal), e os três primeiros dígitos formam um código de área de telefone válido. O tipo de dado para Idades_empregado é um número inteiro entre 15 e 80. Para Nomes_departamento_academico, o tipo de dado é o conjunto de todas as cadeias de caracteres que representam nomes válidos de departamentos. Assim, a um domínio é dado um nome, tipo de dado e formato. Informações adicionais para interpretação dos valores de um domínio também podem ser fornecidas; por exemplo, um domínio numérico como Pesos_pessoa deveria ter uma unidade de medida, como libras ou quilogramas.

Um **esquema de relação** R , indicada por $R(A_1, A_2, \dots, A_n)$, é composto de um nome de relação R e de uma lista de atributos $A_1, A_2, \dots, A_n$. Cada **atributo** A_i é o nome

de um papel desempenhado por algum domínio D no esquema de relação R . D é chamado **domínio** de A_i e é indicado por $\text{dom}(A_i)$. Um esquema de relação é usado para *descrever* uma relação; R é chamado **nome** de sua relação. O **grau** (ou **arity**) de uma relação é o número n de atributos de seu esquema de relação.

Um exemplo de um esquema de relação para uma relação de grau sete, que descreve os universitários, é o seguinte:

ALUNO (Nome, SSN, FoneResidencia, Endereco, FoneEscritorio, Idade, MPG)

Usando o tipo de dado de cada atributo, a definição, algumas vezes, é escrita como:

ALUNO (Nome: *string*, SSN: *string*, FoneResidencia: *string*, Endereco: *string*, FoneEscritorio: *string*, Idade: *integer*, MPG: *real*)

Para esse esquema de relação, ALUNO é o nome da relação, que tem sete atributos. Na definição anterior, mostramos a designação de tipos genéricos, como *string* ou *integer* para os atributos. Mais precisamente, podemos especificar os seguintes domínios definidos previamente para alguns dos atributos da relação ALUNO: $\text{dom}(\text{Nome}) = \text{Nomes}$; $\text{dom}(\text{SSN}) = \text{Numeros_seguro_social}$; $\text{dom}(\text{FoneResidencia}) = \text{Numeros_fone_local}$, $\text{dom}(\text{FoneEscritorio}) = \text{Numeros_fone_local}$, e $\text{dom}(\text{MPG}) = \text{Medias_pontos_graduacao}$. Também é possível referir-se a atributos de um esquema de relação por sua posição na relação, assim, o segundo atributo da relação ALUNO é SSN, enquanto o quarto atributo é Endereco.

Uma **relação** (ou **estado da relação**) r do esquema de relação $R(A_1, A_2, \dots, A_n)$, indicado por $r(R)$, é um conjunto de n -tuplas $r = \{t_1, t_2, \dots, t_m\}$. Cada **n -tupla** t é uma lista ordenada de n valores $t = \langle v_1, v_2, \dots, v_n \rangle$, em que cada valor $v_i, 1 \leq i \leq n$, é elemento do $\text{dom}(A_i)$ ou um valor **null** especial. O $i^{\text{ésimo}}$ valor na tupla t , que corresponde ao atributo A_i , é referido como $t[A_i]$ (ou $t[i]$, se usarmos a notação posicional). Os termos **intenção de relação** para o esquema R e **extensão de relação** para estado de relação $r(R)$ também são comumente usados.

A abaixo mostra um exemplo de uma relação ALUNO que corresponde ao esquema ALUNO já especificado. Cada tupla na relação representa uma entidade aluno em particular. Exibimos a relação como uma tabela, na qual cada tupla é mostrada como uma *linha* e cada atributo corresponde a um *cabeçalho de coluna*, indicando um papel ou interpretação de valores dessa coluna. Os *valores null* representam os atributos cujos valores são desconhecidos ou não existem para alguma tupla individual de ALUNO.

ALUNO	Nome	SSN	FoneResidencia	Endereco	FoneEscritorio	Idade	MPG
	Benjamin Bayer	305-61-2435	373-1616	2918 Bluebonnet Lane	null	19	3.21
	Katherine Ashly	381-62-1245	375-4409	125 Kirby Road	null	18	2.89
	Dick Davidson	422-11-2320	null	3452 Elgin Road	749-1253	25	3.53
	Charles Cooper	489-22-1100	376-9821	265 Lark Lane	749-6492	28	3.93
	Barbara Benson	533-69-1238	839-8461	7384 Fontana Lane	null	19	3.25

Os atributos e as tuplas de uma relação ALUNO.

A definição anterior de uma relação pode ser *redefinida* mais formalmente como segue. Uma relação (ou estado da relação) $r(R)$ é uma **relação matemática** de grau n nos domínios $\text{dom}(A_1), \text{dom}(A_2), \dots, \text{dom}(A_n)$, que é um **subconjunto** do **produto cartesiano** dos domínios que definem R :

$$r(R) \subseteq (\text{dom}(A_1) \times \text{dom}(A_2) \times \dots \times \text{dom}(A_n))$$

O produto cartesiano especifica todas as possíveis combinações de valores dos domínios subjacentes. Então, se indicamos o número total de valores, ou **cardinalidade**, em um domínio D por $|D|$ (presumindo que todos os domínios sejam finitos), o número total de tuplas no produto cartesiano é

$$|\text{dom}(A_1)| \times |\text{dom}(A_2)| \times \dots \times |\text{dom}(A_n)|$$

De todas essas possíveis combinações, um estado de relação em um dado momento, **estado de relação corrente**, reflete apenas as tuplas válidas que representam um estado em particular do mundo real. Em geral, como o estado do mundo real muda, assim também a relação se transforma em outro estado de relação. Entretanto, o esquema R é relativamente estático e *não* muda, exceto muito raramente, por exemplo, por meio da adição de um atributo para representar uma nova informação que, originalmente, não era armazenada na relação.

É possível que muitos atributos possuam o *mesmo domínio*. Os atributos indicam diferentes **papéis**, ou interpretações para um domínio. Por exemplo, na relação ALUNO, o mesmo domínio `Numero_fone_local` desempenha o papel de `FoneResidencia`, referindo-se ao ‘telefone residencial do aluno’, e o papel de `FoneEscritorio`, ao ‘fone do escritório do aluno.’

3.1.2 Características das Relações

A definição anterior de relações implica certas características que fazem uma relação diferente de um arquivo ou de uma tabela. Discutiremos, agora, algumas dessas características.

Ordenação de Tuplas em uma Relação. Uma relação é definida como um *conjunto* de tuplas. Matematicamente, os elementos de um subconjunto *não* têm *ordem* entre eles, portanto, as tuplas em uma relação não têm qualquer ordem em particular. Entretanto, em um arquivo, os registros são fisicamente armazenados em disco (ou na memória), então, sempre há uma ordem entre os registros. Essa ordenação indica o primeiro, o segundo, *i-ésimos* e o último registro no arquivo. Analogamente, quando exibimos uma relação em uma tabela, as linhas são exibidas em uma certa ordem.

A ordenação de tupla não é parte da definição de uma relação, porque uma relação se esforça para representar fatos um nível lógico ou abstrato. Muitas ordens lógicas podem ser especificadas em uma relação. Por exemplo, as tuplas da relação ALUNO, apresentada na figura anterior, item 5.1.1, poderiam ser ordenadas logicamente pelos valores de Nome, ou SSN, ou Idade, ou por algum outro atributo. A definição de uma relação não especifica qualquer ordem: *não* há *preferência* por uma ordenação lógica sobre outra. Por isso a relação exibida na figura a seguir é considerada *idêntica* àquela

mostrada na figura anterior, item 5.1.1. Quando uma relação é implementada como um arquivo ou exibida como uma tabela, uma ordenação em particular pode ser especificada nos registros arquivo ou linhas da tabela.

ALUNO	Nome	SSN	FoneResidencia	Endereco	FoneEscritorio	Idade	MPG
	Dick Davidson	422-11-2320	<i>null</i>	3452 Elgin Road	749-1253	25	3.53
	Barbara Benson	533-69-1238	839-8461	7384 Fontana Lane	<i>null</i>	19	3.25
	Charles Cooper	489-22-1100	376-9821	265 Lark Lane	749-6492	28	3.93
	Katherine Ashly	381-62-1245	375-4409	125 Kirby Road	<i>null</i>	18	2.89
	Benjamin Bayer	305-61-2435	373-1616	2918 Bluebonnet Lane	<i>null</i>	19	3.21

A relação *ALUNO*, apresentada no item 5.1.1, com uma ordenação diferente das tuplas.

Ordenação de Valores Dentro de uma Tupla. De acordo com a definição anterior de uma relação, uma *n*-tupla é uma *lista ordenada* de *n* valores, tanto assim que a ordenação de valores em uma tupla, ou seja, dos atributos no esquema da relação, é importante. Entretanto, em nível lógico, a ordem dos atributos e seus valores *não* são tão importantes, enquanto a correspondência entre atributos e valores for mantida.

Quando uma relação é implementada como um arquivo, os atributos são fisicamente ordenados como campos dentro de um registro.

Valores e Nulls nas Tuplas. Cada valor em uma tupla é um valor **atômico**, isto é, não é divisível em componentes dentro da estrutura do modelo relacional básico.

Um importante conceito se refere a **nulls**, que são usados para representar os valores de atributos que podem ser desconhecidos ou não se aplicar a uma tupla. Um valor especial, chamado **null**, é usado para esses casos. Por exemplo, na figura apresentada no item 5.1.1, algumas tuplas dos alunos têm *null* para seus fones de escritório, porque eles não têm um escritório (isto é, o fone do escritório *não se aplica* a esses alunos). Outros alunos dispõem de *null* para o fone residencial, presumivelmente porque eles não possuem um fone residencial ou têm um, mas não o conhecemos (o valor é *desconhecido*). Em geral, podemos ter *diversos significados* para os valores *null*, como 'valor desconhecido', 'valor existe, mas não está disponível' ou 'atributo não se aplica a essa tupla'. Um exemplo desse último tipo de *null* ocorrerá se adicionarmos um atributo *Visto_status* à relação *ALUNO*, que se aplica apenas às tuplas que representam os alunos estrangeiros. É possível planejar diferentes códigos para diferentes significados de valores *null*. A incorporação de diferentes tipos de valores *null* nas operações de modelo relacional (Capítulo 6) tem apresentado dificuldades e está fora do escopo da nossa discussão.

Interpretação (Sentido) de uma Relação. O esquema de relação pode ser interpretado como uma declaração ou um tipo de **asserção**. Por exemplo, o esquema da relação *ALUNO* da figura apresentada no item 5.1.1, declara que, em geral, uma entidade aluno tem um Nome, SSN, FoneResidencia, Endereco, FoneEscritorio, Idade e MPG. Cada tupla na relação pode, então, ser interpretada como um **fato** ou uma instância em particular da asserção. Por exemplo, a primeira tupla da figura apresentada no item 5.1.1 estabelece o fato de que há um aluno cujo nome é Benjamin Bayer, seu SSN é 305-61-2435, sua Idade é 19, e assim por diante.

Observe que algumas relações devem representar fatos sobre *entidades*, ao passo que outras relações devem representar fatos sobre *relacionamentos*. Por exemplo, o esquema de relação CURSO_HAB (SSNALuno,CodigoDepartamento) estabelece que os alunos fazem seus cursos em departamentos acadêmicos. Uma tupla nessa relação liga um aluno ao departamento de seu curso. Por isso o modelo relacional representa os fatos sobre ambos, entidades e relacionamentos, *indistintamente*, como relações. As vezes, esse fato compromete o entendimento, pois é preciso supor se uma relação representa um tipo entidade ou um tipo relacionamento.

3.2 Restrições do Modelo Relacional e Esquemas de um Banco de Dados Relacional

Até aqui discutimos as características de uma única relação. Em um banco de dados relacional, normalmente existirão muitas relações, e as tuplas dessas relações estão, em geral, relacionadas de várias maneiras. O estado do banco de dados como um todo corresponderá aos estados de todas as suas relações em um determinado instante. Há, geralmente, muitas limitações ou restrições para os valores reais em um estado do banco de dados. Essas restrições são derivadas de regras do minimundo que o banco de dados representa.

Nesta seção discutiremos as várias limitações nos dados que podem ser especificadas em um banco de dados relacional, na forma de restrições. As restrições em bancos de dados podem, geralmente, ser divididas em três categorias principais:

1. Restrições que são inerentes ao modelo de dado. Vamos chamá-las **restrições inerentes baseadas em modelo**.
2. Restrições que podem ser expressas diretamente nos esquemas do modelo de dado, normalmente por suas especificações em DDL (linguagem de definição de dado, *data definition language*). São conhecidas como **restrições baseadas em esquema**.
3. Restrições que *não podem* ser expressas diretamente nos esquemas do modelo de dado e, por isso, devem ser expressas e impostas pelos programas de aplicação. Chamamo-nas de **restrições baseadas em aplicação**.

As características das relações, que discutimos no item 5.1.2, são as restrições inerentes do modelo relacional e pertencem à primeira categoria; por exemplo, a restrição de que uma relação não pode ter tuplas repetidas é aquela inerente. As restrições que discutiremos neste item são da segunda categoria, daquelas que podem ser expressas no esquema do modelo relacional por meio de DDL. As restrições da terceira categoria são mais gerais e difíceis de expressar e impor dentro do modelo de dados e, por isso, são geralmente asseguradas por meio dos programas de aplicação.

Discutiremos, agora, os tipos principais de restrições que podem ser expressas no modelo relacional, as restrições baseadas em esquema, da segunda categoria. Incluem as restrições de domínio, restrições de chave, restrições em null, restrições de integridade de entidade e restrições de integridade referencial.

3.2.1 Restrições de Domínio

As restrições de domínio especificam que, dentro de cada tupla, o valor de cada atributo A deve ser um valor atômico do domínio $\text{dom}(A)$. Já discutimos, no item 5.1.1, as formas pelas quais os domínios podem ser especificados. Os tipos de dados associados aos domínios incluem os tipos de dados numéricos padrões para inteiros (como inteiro curto, inteiro e inteiro longo) e números reais (ponto flutuante e flutuante de precisão dupla). Os caracteres, booleanos e as cadeias de caracteres de comprimento fixo e de comprimento variável também estão disponíveis, como *data*, *hora*, *timestamp* (marca de tempo) e, em alguns casos, os tipos de dado de moeda. Outros domínios possíveis podem ser descritos por um subconjunto de valores de um tipo de dado ou por um tipo de dado enumerado, por meio dos quais todos os possíveis valores serão, explicitamente, relacionados.

3.2.2 Restrições de Chave e em Valores Null

Uma *relação* é definida como um *conjunto de tuplas*. Por definição, todos os elementos de um conjunto são distintos, por isso, todas as tuplas da relação também devem ser distintas. Isso significa que duas tuplas não podem ter a mesma combinação de valores para *todos* os seus atributos. Geralmente, há outros **subconjuntos de atributos** de um esquema de relação R com a propriedade de que duas tuplas, em qualquer estado de relação r de R , não tenham as mesmas combinações de valores para esses atributos.

Qualquer um desses conjuntos de atributos é chamado **superchave** do esquema de relação R . Uma superchave especifica uma *restrição de unicidade*, na qual duas tuplas distintas, em qualquer estado r de R , não podem ter o mesmo valor. Toda relação tem, pelo menos, uma superchave *default*, o conjunto de todos os seus atributos. Uma superchave pode ter os atributos redundantes, entretanto, um conceito mais vantajoso é o de uma *chave* que não apresenta redundância. Uma **chave** K de um esquema de relação R é uma superchave de R , com a propriedade adicional de, ao remover qualquer atributo A de K , o conjunto de atributos K' restante não será mais uma superchave de R . Portanto, uma chave satisfaz duas restrições:

1. Duas tuplas distintas, em qualquer estado da relação, não podem ter valores idênticos para (todos) os atributos da chave.
2. Ela é uma *superchave mínima*, isto é, uma superchave da qual não podemos remover quaisquer atributos e ainda manter a restrição de unicidade garantida pela condição 1.

A primeira condição se aplica tanto às chaves como às superchaves. A segunda condição é requerida apenas para as chaves. Por exemplo, considere a relação ALUNO da figura apresentada no item 5.1.1. O conjunto de atributo {SSN} é uma chave de ALUNO, porque duas tuplas de aluno não podem ter o mesmo valor para SSN. Todo conjunto de atributos que contiver SSN por exemplo, {SSN, Nome, Idade} será uma superchave. Entretanto, a superchave {SSN, Nome, Idade} não é uma chave de ALUNO, pois remover Nome ou Idade, ou ambos, do conjunto, ainda nos deixa com uma superchave. Em geral, toda superchave, formada por um único atributo, também é uma chave. Uma chave com vários atributos exige que *todos* garantam a propriedade de

unicidade.

O valor de um atributo-chave também pode ser usado para identificar, unicamente, cada tupla na relação. Por exemplo, o valor 305-61-2435 de SSN identifica unicamente a tupla correspondente a Benjamin Bayer na relação ALUNO. Observe que um conjunto de atributos constituindo uma chave é uma propriedade do esquema da relação; é uma restrição que se manteria em *todos* os estados válidos de relação desse esquema. Uma chave é determinada em função do significado dos atributos, e a propriedade não *varia no tempo*: ela deve continuar a ser mantida quando inserimos novas tuplas na relação. Por exemplo, não podemos e não deveríamos indicar o atributo Nome, da relação ALUNO na figura apresentada no item 5.1.1, como uma chave, pois é possível existirem dois alunos com nomes idênticos, a qualquer momento, em um estado válido.

Em geral, um esquema de relação pode ter mais de uma chave. Nesse caso, cada uma das chaves é chamada **chave candidata**. Por exemplo, a relação CARRO, na figura apresentada abaixo, tem duas chaves candidatas: NumeroLicenca e NumeroChassi. É comum indicar uma das chaves candidatas como a **chave primária** da relação. Essa é a chave candidata cujos valores são usados para *identificar* tuplas na relação. Usaremos a convenção de que os atributos que formam a chave primária de um esquema de relação devam ser sublinhados, como mostrado na figura abaixo. Observe que, quando um esquema de relação tem diversas chaves candidatas, a escolha de uma para se tornar a chave primária é arbitrária, entretanto, geralmente é melhor optar por uma chave primária com um atributo único ou com um número pequeno de atributos.

CARRO	<u>NumeroLicenca</u>	NumeroChassi	Marca	Modelo	Ano
	Texas ABC-739	A69352	Ford	Mustang	96
	Flórida TVP-347	B43696	Oldsmobile	Cutlass	99
	Nova York MPO-22	X83554	Oldsmobile	Delta	95
	Califórnia 432-TFY	C43742	Mercedes	190-D	93
	Califórnia RSK-629	Y82935	Toyota	Camry	98
	Texas RSK-629	U028365	Jaguar	XJS	98

A relação CARRO com duas chaves candidatas: NumeroLicenca e NumeroChassi.

Outra restrição nos atributos especifica se valores *null* são ou não permitidos. Por exemplo, se toda tupla de ALUNO tem um valor válido, não nulo, para o atributo Nome, então Nome de ALUNO é forçado a ser **NOT NULL** (não nulo).

3.2.3 Bancos de Dados Relacionais e Esquemas de um Banco de Dados Relacional

As definições e restrições que temos discutido até aqui se aplicam às relações únicas e seus atributos. Um banco de dados relacional contém, geralmente, muitas relações, com suas tuplas relacionadas de várias maneiras entre si. Neste item, definimos um banco de dados e um esquema de um banco de dados relacional. Um **esquema de um banco de dados relacional** S é um conjunto de esquemas de relação $S = \{R_1, R_2, \dots, R_m\}$ e um conjunto de **restrições de integridade** IC. Um **estado de um banco de dados**

relacional DB de S é um conjunto dos estados da relação $DB = \{r_1, r_2, \dots, r_m\}$, de forma que cada r_i seja um estado de R_i , e de maneira que os estados da relação r_i satisfaçam as restrições de integridade especificadas em IC. A figura (a) apresentada abaixo, mostra um esquema de um banco de dados relacional, que chamamos $EMPRESA = \{EMPREGADO, DEPARTAMENTO, DEPTO_LOCALIZACOES, PROJETO, TRABALHA_EM, DEPENDENTE\}$. Os atributos sublinhados representam as chaves primárias. A figura (b) apresentada na página seguinte, mostra um estado do banco dados relacional, correspondendo ao esquema EMPRESA. Quando nos referimos a um banco de dados relacional incluímos, implicitamente, seu esquema e seu estado corrente. Um estado de um banco de dados que não obedece a todas as restrições de integridade é chamado **estado inválido**, e um estado que satisfaz todas as restrições em IC é conhecido como **estado válido**.

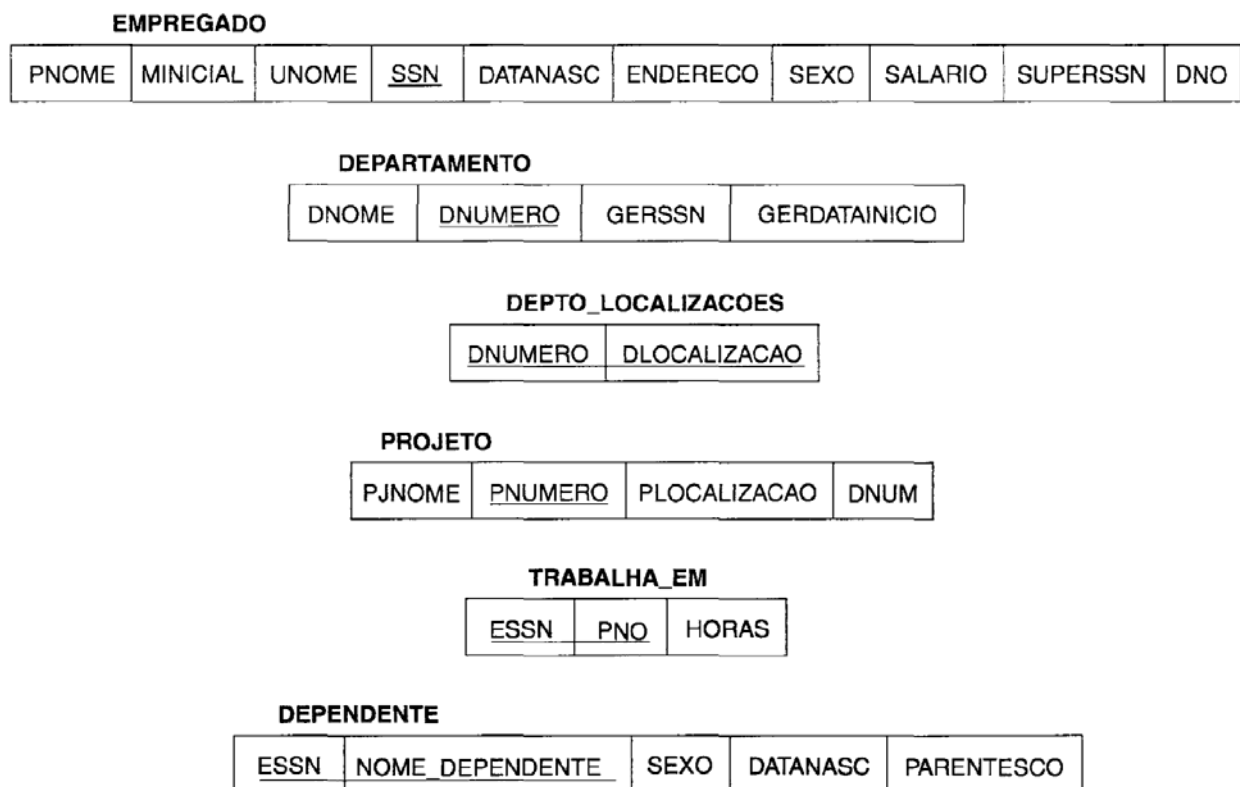


Figura (a) – Diagrama para o esquema do banco de dados relacional EMPRESA.

Cada SGBD relacional deve ter uma linguagem de definição de dados (DDL) para a definição de um esquema de um banco de dados relacional. Os SGBDs atuais estão, em sua maior parte, usando o SQL com esse objetivo.

As restrições de integridade são especificadas no esquema do banco de dados, e espera-se que sejam mantidas em qualquer estado válido do banco de dados desse esquema. Além das restrições de domínio, chave e NOT NULL, outros dois tipos de restrições são considerados parte do modelo relacional: de integridade de entidade e de integridade referencial.

EMPREGADO	PNOME	MINICIAL	UNOME	SSN	DATANASC	ENDEREÇO	SEXO	SALARIO	SUPERSSN	DNO
	John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
	Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
	Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
	Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
	Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
	Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
	Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
	James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	null	1

DEPT_LOCALIZACOES	DNUMERO	DLOCALIZACAO
	1	Houston
	4	Stafford
	5	Bellaire
	5	Sugarland
		Houston

DEPARTAMENTO	DNOME	DNUMERO	GERSSN	GERDATAINICIO
	Pesquisa	5	333445555	1988-05-22
	Administração	4	987654321	1995-01-01
	Sede administrativa	1	888665555	1981-06-19

TRABALHA_EM	ESSN	PNO	HORAS
	123456789	1	32.5
	123456789	2	7.5
	666884444	3	40.0
	453453453	1	20.0
	453453453	2	20.0
	333445555	2	10.0
	333445555	3	10.0
	333445555	10	10.0
	333445555	20	10.0
	999887777	30	30.0
	999887777	10	10.0
	987987987	10	35.0
	987987987	30	5.0
	987654321	30	20.0
	987654321	20	15.0
	888665555	20	null

PROJETO	PJNOME	PNUMERO	PLOCALIZACAO	DNUM
	ProdutoX	1	Bellaire	5
	ProdutoY	2	Sugarland	5
	ProdutoZ	3	Houston	5
	Automatização	10	Stafford	4
	Reorganização	20	Houston	1
	Novos Benefícios	30	Stafford	4

DEPENDENTE	ESSN	NOME_DEPENDENTE	SEXO	DATANASC	PARENTESCO
	333445555	Alice	F	1986-04-05	FILHA
	333445555	Theodore	M	1983-10-25	FILHO
	333445555	Joy	F	1958-05-03	CÔNJUGE
	987654321	Abner	M	1942-02-28	CÔNJUGE
	123456789	Michael	M	1988-01-04	FILHO
	123456789	Alice	F	1988-12-30	FILHA
	123456789	Elizabeth	F	1967-05-05	CÔNJUGE

Figura (b) – Um estado de um banco de dados possível para o esquema do banco de dados relacional EMPRESA.

3.2.4 Integridade de Entidade, Integridade Referencial e Chaves Estrangeiras

A **restrição de integridade de entidade** estabelece que nenhum valor de chave primária pode ser *null*. Isso porque o valor da chave primária é usado para identificar as tuplas individuais em uma relação. Ter valores *null* para a chave primária implica não podermos identificar alguma tupla. Por exemplo, se duas ou mais tuplas tiverem *null* em suas chaves primárias, poderemos não ser capazes de distingui-las, se tentarmos fazer referência a elas por intermédio de outras relações.

As restrições de chave e as de integridade de entidade são especificadas na própria relação. A **restrição de integridade referencial** é classificada entre duas relações e é usada para manter a consistência entre as tuplas nas duas relações. Informalmente, a restrição de integridade referencial declara que uma tupla em uma relação, que faz referência a outra relação, deve se referir a uma *tupla existente* nessa relação. Por exemplo, na figura (b), apresentada na página anterior, o atributo DNO de EMPREGADO fornece o número do departamento para o qual cada empregado trabalha, portanto, seu valor em toda tupla de EMPREGADO deve corresponder ao valor do DNUMERO de alguma tupla na relação DEPARTAMENTO.

Para definir integridade referencial mais formalmente definimos, primeiro, o conceito de uma *chave estrangeira*. As condições dadas a seguir para uma chave estrangeira especificam uma restrição de integridade referencial entre os dois esquemas de relação R_1 e R_2 . Um conjunto de atributos FK do esquema da relação R_1 é uma **chave estrangeira** de R_1 , que faz **referência** à relação R_2 , se ele satisfizer as duas regras seguintes:

1. Os atributos de FK têm o(s) mesmo(s) domínio(s) que os atributos da chave primária PK de R_2 ; diz-se que os atributos de FK fazem **referência** ou se **referem** à relação R_2 .
2. Um valor de FK em uma tupla t_1 do estado corrente $r_1(R_1)$ ou ocorre como um valor de PK para alguma tupla t_2 no estado corrente $r_2(R_2)$ ou é *null*. No caso anterior, temos $t_1[\text{FK}] = t_2[\text{PK}]$, e dizemos que a tupla t_1 faz referência ou se **refere** à tupla t_2 .

Nessa definição, R_1 é chamada **relação referência**, e R_2 é a **relação referida**. Se essas duas condições forem asseguradas, uma **restrição de integridade referencial** de R_1 para R_2 é dita garantida. Em um banco de dados com muitas relações, geralmente há muitas restrições de integridade referencial.

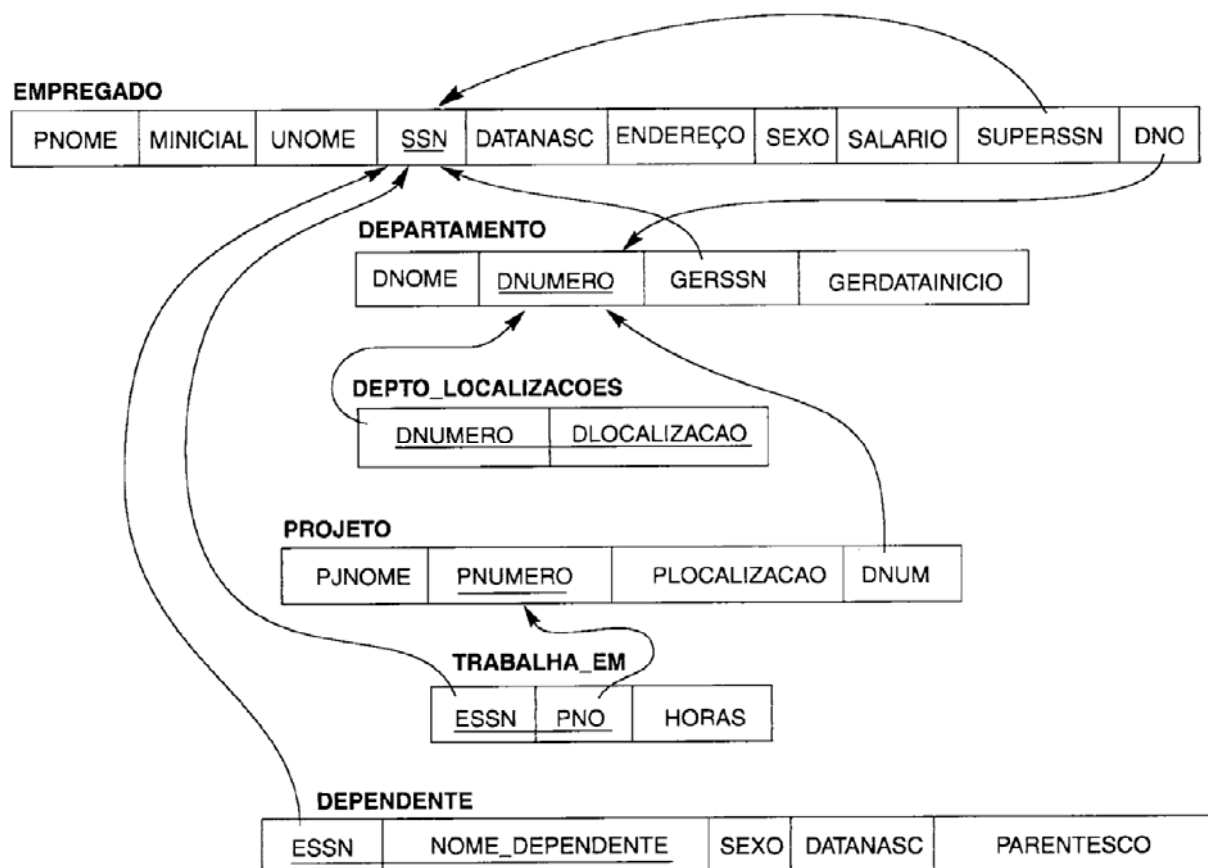
Para especificar essas restrições devemos, primeiro, ter um entendimento claro do significado, ou do papel, que cada conjunto de atributos desempenha nos vários esquemas de relação do banco de dados. As restrições de integridade referencial surgem dos *relacionamentos entre as entidades* representadas em esquemas de relação. Por exemplo, considere o banco de dados mostrado na figura (b) da página anterior. Na relação EMPREGADO, o atributo DNO refere-se ao departamento para o qual um empregado trabalha, portanto, indicamos DNO como chave estrangeira de EMPREGADO, fazendo referência à relação DEPARTAMENTO. Isso significa que um valor de DNO em qualquer tupla t_1 da relação EMPREGADO deve corresponder a um valor da chave primária de DEPARTAMENTO, o atributo DNUMERO, em alguma tupla t_2 , da relação DEPARTAMENTO, ou o valor de DNO *pode* ser *null* se o empregado não pertencer a um departamento. Na figura (b) da página anterior, a tupla para o empregado 'John Smith' faz referência à tupla para o departamento 'Pesquisa', indicando que 'John Smith' trabalha para esse departamento.

Observe que uma chave estrangeira pode *se referir à sua própria relação*. Por exemplo, o atributo SUPERSSN em EMPREGADO refere-se ao supervisor de um empregado; esse é outro empregado, representado por uma tupla na relação EMPREGADO. Portanto, SUPERSSN é uma chave estrangeira que faz referência, ela mesma, à relação EMPREGADO. Na figura (b) da página anterior, a tupla do empregado 'John Smith' faz referência à tupla do empregado 'Franklin Wong', indicando que 'Franklin Wong' é o supervisor de 'John Smith'.

Podemos *exibir diagramaticamente as restrições de integridade referencial* puxando um arco direto de cada chave estrangeira à relação que ela faz referência. Para

maior clareza, a seta deve apontar para a chave primária da relação referida. A figura apresentada a seguir, mostra o esquema da figura (a), apresentada na página 49, com as restrições de integridade referencial exibidas dessa maneira.

Todas as restrições de integridade deveriam ser especificadas no esquema do banco de dados relacional, caso queiramos impor essas restrições aos estados do banco de dados. Portanto, a DDL possui recursos para especificar os vários tipos de restrições e, assim, o SGBD pode, automaticamente, garanti-las. A maioria dos SGBDs relacionais suporta restrições de chave e de integridade de entidade, e providenciam suporte para a integridade referencial. Essas restrições são especificadas como parte da definição de dados.



Restrições de integridade referencial exibidas no esquema de um banco de dados relacional EMPRESA.

3.2.5 Outros Tipos de Restrições

As restrições de integridade anteriores não incluem uma grande classe de restrições genéricas chamadas, às vezes, restrições de *integridade semântica*, que podem ser especificadas e impostas em um banco de dados relacional. Os exemplos dessas restrições são 'o salário de um empregado não deveria exceder o do supervisor do empregado' e 'o número máximo de horas que um empregado pode trabalhar por semana, em todos os projetos, é 56'. Essas restrições podem ser especificadas e

impostas dentro dos programas de aplicação que atualizam o banco de dados ou pelo uso de uma **linguagem de especificação de restrição** de propósito geral.

Os tipos de restrições discutidas até aqui podem ser chamados restrições de estado, porque definem as restrições que um *estado válido* do banco de dados deve satisfazer. Outro tipo de restrição, conhecido como restrições de transição, pode ser estabelecido para lidar com as mudanças de estado no banco de dados. Um exemplo de uma restrição de transição é: 'o salário de um empregado pode ser apenas aumentado'. Essas restrições geralmente são impostas pelos programas de aplicação.

3.3 Operações de Atualização e Tratamento de Violações de Restrição

As operações do modelo relacional podem ser categorizadas em *recuperações* e *atualizações*. As operações da álgebra relacional, que podem ser usadas para especificar as **recuperações**, serão discutidas em detalhes na próxima unidade. Uma expressão da álgebra relacional forma uma nova relação depois de aplicar um número de operadores algébricos a um conjunto existente de relações; seu uso principal é para consultar um banco de dados. O usuário formula uma consulta que especifica o dado de interesse e uma nova relação é formada pela aplicação de operadores relacionais para recuperar esse dado. Essa relação se torna a resposta à consulta do usuário. A unidade 4 também introduz a linguagem chamada cálculo relacional, que é usada para definir, declarativamente, a nova relação, sem dar uma ordem específica das operações.

Neste item concentramo-nos nas operações para a **modificação** ou **atualização** no banco de dados. Há três operações básicas de atualização em relações: *insert*, *delete* e *modify*. **Insert** é usada para inserir uma nova tupla ou tuplas em uma relação. **Delete** é empregada para remover as tuplas. E **Update** (ou **Modify**) é utilizada para mudar os valores de alguns atributos em tuplas existentes. Quando essas operações são aplicadas, as restrições de integridade especificadas no esquema do banco de dados relacional não devem ser violadas. Neste item, discutiremos os tipos de restrições que podem ser violadas em cada operação de atualização e os tipos de ações que devem ser tomadas se uma atualização causar uma violação. Usamos o banco de dados mostrado na figura (b) *Um estado de um banco de dados possível para o esquema do banco de dados relacional EMPRESA*, apresentado no item 3.2.3 da apostila, para exemplos e discutimos apenas as restrições de chave, de integridade de entidade, bem como as restrições de integridade referencial, mostradas na figura apresentada no item 3.2.4 da apostila sobre *restrições de integridade referencial exibidas no esquema de um banco de dados relacional EMPRESA*. Para cada tipo de atualização damos algumas operações de exemplo e discutimos as restrições que cada operação pode violar.

3.3.1 A Operação Insert

A operação **Insert** fornece uma lista de valores de atributos para uma nova tupla t , que é inserida em uma relação R . **Insert** pode violar qualquer um dos quatro tipos de restrições discutidas na seção anterior. As restrições de domínio podem ser violadas se um valor de atributo que é dado não pertencer ao domínio correspondente. Já as restrições de chave podem ser violadas se um valor de chave, em uma nova tupla t , já existir em outra tupla da relação $r(R)$. A integridade de entidade pode ser violada se a

chave primária da nova tupla *t* for *null*. A integridade referencial pode ser violada se o valor de qualquer chave estrangeira em *t* referir-se a uma tupla que não existe na relação referida. Aqui estão alguns exemplos para ilustrar a discussão.

1. Inserir <'Cecilia', 'F', 'Kolonsky', *null*, '1960-04-05', '6357 Windy Lane, Katy, TX', F, 28000, *null*, 4> em EMPREGADO.
 - Essa inserção viola a restrição de integridade de entidade (*null* para a chave primária SSN), assim, ela será rejeitada.
2. Inserir <'Alicia', 'J', 'Zelaya', '999887777', '1960-04-05', '6357 Windy Lane, Katy, TX', F, 28000, '987654321', 4> em EMPREGADO.
 - Essa inserção viola a restrição de chave, porque outra tupla com o mesmo valor de SSN já existe na relação EMPREGADO, então ela será rejeitada.
3. Inserir <'Cecilia', 'F', 'Kolonsky', '677678989', '1960-04-05', '6357 Windswept, Katy, TX', F, 28000, '987654321', 7> em EMPREGADO.
 - Essa inserção viola a restrição de integridade referencial especificada em DNO, porque não existe tupla de DEPARTAMENTO com DNUMERO = 7.
4. Inserir <'Cecilia', 'F', 'Kolonsky', '677678989', '1960-04-05', '6357 Windy Lane, Katy, TX', F, 28000, *nuli*, 4> em EMPREGADO.
 - Essa inserção satisfaz todas as restrições, assim, ela é aceitável.

Se uma inserção violar uma ou mais restrições, a opção *default* é rejeitar a *inserção*. Nesse caso, seria útil se o SGBD explicasse ao usuário por que a inserção foi rejeitada. Outra opção é tentar *corrigir a razão para a rejeição da inserção*, mas isso não é comum para as violações causadas por Insert; ela é preferencialmente usada com mais frequência na correção de violações para *Delete* e *Update*. Na operação 1 acima, o SGBD poderia solicitar ao usuário um valor para SSN, e poderia aceitar a inserção se um valor válido para SSN fosse fornecido. Na operação 3, o SGBD também poderia solicitar ao usuário que mudasse o valor de DNO para um valor válido (ou determiná-lo como *null*), ou poderia pedir ao usuário que inserisse uma tupla de DEPARTAMENTO com DNUMERO = 7, aceitando a primeira inserção somente depois que essa operação fosse aceita. Observe que, nesse último caso, a violação de inserção pode atingir a relação EMPREGADO após a **cascata**, caso o usuário tente inserir uma tupla para o departamento 7 com um valor de GERSSN que não exista na relação EMPREGADO.

3.3.2 A Operação *Delete*

A operação **Delete** pode violar apenas a integridade referencial se a tupla removida for referida por chaves estrangeiras de outras tuplas no banco de dados. Para especificar a remoção, uma condição nos atributos da relação seleciona a tupla (ou tuplas) a ser removida. Eis alguns exemplos.

1. Remover a tupla TRABALHA_EM com ESSN = '999887777' e PNO = 10.
 - Essa remoção é aceitável.
2. Remover a tupla EMPREGADO com SSN = '999887777'.

- Essa remoção não é aceitável, porque há tuplas em TRABALHA_EM que se referem a essa tupla. Portanto, se a tupla for removida, vão resultar violações de integridade referencial.
3. Remover a tupla EMPREGADO com SSN = '333445555'.
 - Essa remoção resultará em violações de integridade referencial piores ainda, porque a tupla envolvida é referida por tuplas das relações EMPREGADO, DEPARTAMENTO, TRABALHA_EM e DEPENDENTE.

Há diversas opções disponíveis caso uma operação de remoção cause violação. A primeira opção é *rejeitar a remoção*. A segunda é a *remoção em cascata* (ou *propagação*), eliminando as tuplas que fazem referência à tupla que está sendo removida. Por exemplo, na operação 2, o SGBD poderia remover automaticamente as tuplas atingidas de TRABALHA_EM com SSN = '999887777'. Uma terceira opção é *modificar os valores dos atributos de referência* que causam a violação; cada um desses valores ou seria determinado por *null* ou modificado para referir outra tupla válida. Observe que, se um atributo de referência que cause violação fizer *parte da chave primária*, ele não poderá ser determinado como *null*; de outra maneira, ele violaria a integridade da entidade.

As combinações dessas três opções também são possíveis. Por exemplo, para evitar que a operação 3 cause uma violação, o SGBD pode remover automaticamente todas as tuplas de TRABALHA_EM e DEPENDENTE com ESSN = '333445555'. As tuplas em EMPREGADO, com SUPERSSN = '333445555', e a tupla em DEPARTAMENTO, com GERSSN = '333445555', podem ter seus valores de SUPERSSN e GERSSN mudados para outros valores válidos ou para *null*. Embora faça sentido remover automaticamente as tuplas de TRABALHA_EM e DEPENDENTE, que se referem a uma tupla de EMPREGADO, não deve fazer sentido remover outras tuplas de EMPREGADO ou uma tupla de DEPARTAMENTO.

3.3.3 A Operação Update

A operação **Update** (ou **Modify**) é usada para mudar os valores de um ou mais atributos em uma tupla (ou tuplas) de alguma relação *R*. É necessário especificar uma condição nos atributos da relação para selecionar a tupla (ou tuplas) a ser modificada. Aqui estão alguns exemplos.

1. Modificar o SALARIO da tupla de EMPREGADO com SSN = '999887777' para 28000.
 - Aceitável.
2. Modificar o DNO da tupla de EMPREGADO com SSN = '999887777' para 1.
 - Aceitável.
3. Modificar o DNO da tupla de EMPREGADO com SSN = '999887777' para 7.
 - Inaceitável, porque viola a integridade referencial.
4. Modificar o SSN da tupla de EMPREGADO com SSN = '999887777' para '987654321'.
 - Inaceitável, pois viola as restrições de chave primária e as de integridade referencial.

Mudar um atributo que não é nem uma chave primária nem uma chave estrangeira geralmente não causa problemas; o SGBD precisa apenas verificar e confirmar que o novo valor é do tipo de dado e de domínio correto. Modificar um valor de chave primária é similar a remover uma tupla e inserir outra em seu lugar, porque usamos a chave primária para identificar as tuplas. Portanto, as questões discutidas anteriormente, nos itens 3.3.1 (*Insert*) e 3.3.2 (*Delete*), entram em execução. Se um atributo de chave estrangeira for modificado, o SGBD deverá se certificar de que o novo valor refere-se a uma tupla existente na relação referida (ou é *null*). Existem opções similares para tratar as violações de integridade referencial causadas por *Update*, como aquelas opções discutidas para a operação *Delete*.